

CDC — Clock Domain Crossing

📌 서로 다른(비동기) 클럭 도메인 사이로 신호를 안전하게 전달해 메타스터빌리티 전파를 막는 설계 기법 · 📌 P1, 지원서연계 · 🧠 메모리 브릿지: 메모리 컨트롤러 코어 clock ↔ DRAM/PHY clock(DFI), HBM Base Die의 host-logic ↔ DRAM-stack 경계가 전부 CDC. SFR(CSR) 설정값이 PHY/datapath clock으로 넘어가는 길도 CDC다.

핵심 개념

- **메타스터빌리티**: setup/hold 위반 시 FF 출력이 0/1 사이 불확정 상태로 떠 있다가 확률적으로 resolve. 비동기 도메인 경계에서 sampling 타이밍을 보장 못 하므로 필연.
- **MTBF** = $e^{(t_r/\tau)} / (T_0 \cdot f_c \cdot f_d)$. t_r =resolve에 허용된 시간(settling), τ =resolution time constant(현대 CMOS ~20-50ps), T_0 =metastability window, f_c =sampling clock, f_d =입력 toggle rate. t_r 에 **지수적** 의존 → FF 하나 더 달면 MTBF 폭발적 개선.
- **2-FF 동기화**: 목적지 도메인 FF 2단 직렬. 1단이 metastable 돼도 거의 한 클럭 주기(t_r)를 벌여 2단 전에 resolve. latency +1~2 cycle. **1-bit, level 신호에만** 안전.
- **multi-bit 위험 = bit skew**: 버스 비트마다 도착/resolve 시각이 달라 동기화 순간 일시적으로 잘못된 조합값이 보임. → 비트별 2-FF 금지.
- **Gray code**: 인접 값이 1비트만 변함 → skew 있어도 "옛값 or 새값"만 보이고 중간 invalid 없음. FIFO 포인터 동기화 핵심.
- **Async FIFO**: dual-clock RAM + wr_ptr(쓰기 도메인)/rd_ptr(읽기 도메인). 포인터를 Gray로 바꿔 반대 도메인에 2-FF 동기화 후 비교 → full/empty. 포인터에 1bit 추가(MSB)로 full/empty 구별.
- **Handshake(req/ack)**: req로 데이터 안정 유지, 수신측이 동기화·캡처 후 ack, 송신측이 ack 동기화 후 req 해제(4-phase). robust하지만 round-trip latency로 느림.
- **MUX-enable/recirculation**: enable(1-bit) 만 동기화, 데이터 버스는 그대로 두되 enable로 load. 데이터는 false-path지만 캡처 순간 안정 보장.
- **Reconvergence**: 따로 동기화된 여러 신호가 재합류 → 도착 cycle 달라 glitch. 한 번만 동기화 / gray / handshake로 회피.
- **정적 검증**: SpyGlass CDC, Questa CDC, VC SpyGlass. 구조(crossing 탐지·동기화기 유무)+기능(프로토콜 assertion). 안전 crossing은 **waiver**.

예상 면접 Q&A

Q1. 메타스터빌리티가 무엇이고 왜 CDC에서 반드시 발생하나요?

A. FF의 setup/hold 윈도우 안에서 입력이 변하면 출력이 Vdd/2 근처 불확정 상태로 떠 있다가 양의 피드백으로 확률적으로 0 또는 1로 resolve됩니다. 비동기 도메인 경계에서는 송신 데이터와 수신 클럭이 위상 관계가 없어 언젠가는 반드시 윈도우 안에서 변하므로 메타스터빌리티를 **0으로 만들 수 없고**, 확률(MTBF)을 충분히 크게 만드는 게 목표입니다. - L, 꼬리질문: 한 도메인 안(동기)에서는 왜 안 생기나? → 모든 FF가 같은 clock이라 STA로

setup/hold를 정적 보장하기 때문. CDC는 그 보장이 불가능. - 🧠 메모리 브릿지: 메모리 컨트롤러 코어 clock과 PHY clock은 위상 무관 → command/data가 경계 넘을 때 본질적으로 metastable risk.

Q2. MTBF 공식과 이를 개선하는 방법은?

A. $MTBF = e^{(t_r/\tau)} / (T_0 \cdot f_c \cdot f_d)$. t_r 은 메타 resolve에 주어진 시간, τ 는 FF의 resolution 시정수, T_0 는 metastability window, f_c 는 sampling clock, f_d 는 입력 toggle 빈도입니다. t_r 에 **지수적으로** 의존하는 게 핵심이라, FF를 한 단 더 추가(2-FF→3-FF)하거나 sampling clock을 늦추면 t_r 이 늘어 MTBF가 수십~수만 배 좋아집니다. 반대로 고주파일수록 $f_c \cdot f_d$ 가 커져 MTBF가 급격히 나빠집니다. - L, 꼬리질문: 왜 3단이 항상 답은 아닌가? → latency가 높고, 보통 2단으로 충분한 MTBF(수년~수천년)가 나오기 때문. 초고주파/안전등급에서만 3단. - 🧠 메모리 브릿지: HBM/GDDR처럼 f 가 매우 높은 인터페이스일수록 동일 단수라도 MTBF가 뽀뽀 → 동기화기 설계 마진을 더 본다.

Q3. 2-FF 동기화기가 어떻게 동작하고 왜 2개인가요?

A. 수신 도메인 clock으로 동작하는 FF 2개를 직렬로 둡니다. 1단 FF가 metastable 해도 2단이 캡처하기 전 거의 한 clock 주기(t_r) 동안 resolve할 시간을 벌여 줍니다. 그 결과 2단 출력이 metastable일 확률이 지수적으로 작아집니다. 대신 1~2 cycle latency가 추가되고, **반드시 1-bit-level 신호에만** 써야 합니다(다비트는 bit skew). - L, 꼬리질문: 두 FF 사이엔 조합 로직을 넣어도 되나? → 안 됨. 1단의 resolve 시간을 갉아먹어 MTBF가 깨짐. 반드시 back-to-back. - 🧠 메모리 브릿지: PHY status flag(예: cal done, lock)를 코어로 올릴 때 전형적으로 2-FF.

Q4. 멀티비트 버스를 비트별 2-FF로 동기화하면 왜 위험한가요?

A. 비트마다 송신측 도착 시각과 수신측 metastable resolve 방향이 달라, 동기화되는 순간 일부 비트는 새 값, 일부는 옛 값으로 잡혀 **실제로는 존재하지 않은 중간 조합값**이 한두 cycle 보일 수 있습니다(bit skew/data coherency 깨짐). 카운터값이 0111→1000 갈 때 0000이나 1111이 순간적으로 보이는 식입니다. 그래서 다비트는 (1)Gray code, (2)async FIFO, (3)handshake/MUX-recirculation 중 하나로 묶어 전달합니다. - L, 꼬리질문: gray로도 안 되는 경우는? → 임의 다비트(연속 증가가 아닌 값)는 gray가 안 통함 → FIFO나 handshake로. - 🧠 메모리 브릿지: address/burst length 같은 다비트 command를 PHY로 넘길 때 bit skew 방지가 필수.

Q5. Gray code가 CDC에서 왜 유용한가요?

A. Gray code는 인접한 값끼리 **딱 1비트만** 바뀝니다. 따라서 비트별로 따로 동기화돼 skew가 생겨도 수신측은 항상 "직전 값" 아니면 "현재 값"만 보고, 절대 invalid한 중간값을 보지 않습니다. 1비트만 변하니 메타 위험 비트도 한 개뿐이라 안전. 그래서 async FIFO의 wr/rd 포인터를 Gray로 변환해 동기화합니다. - L, 꼬리질문: full/empty 비교는 binary로 다시 바뀌어서 하나? → 포인터끼리 Gray 영역에서 직접 비교(특정 비트 반전 규칙)하거나, 동기화 후 binary로 변환해 비교. 핵심은 "전달은 Gray". - 🧠 메모리 브릿지: 비동기 clock 경계의 read/write 카운터 전달에 그대로 적용.

Q6. 비동기 FIFO의 구조와 포인터 동기화를 설명하세요.

A. 양쪽 clock이 접근하는 dual-port RAM과, 쓰기 도메인의 wr_ptr·읽기 도메인의 rd_ptr로 구성됩니다. 각 포인터를 **Gray로 변환**→**반대 도메인에 2-FF 동기화**→**비교**해서 full/empty를 만듭니다. 깊이가 2^n 이면 포인터는 $n+1$ bit로 뒤서 MSB(wrap 비트)로 full(같은 하위, 다른 MSB)과 empty(완전히 같음)를 구분합니다. 데이터

자체는 동기화 안 하고, 포인터가 "쓸 수 있다/읽을 수 있다"를 보장하는 순간에만 접근하므로 안전합니다. - L, 꼬리질문: empty/full 판정이 보수적인 이유? → 동기화 latency 때문에 상대 포인터가 "과거값"으로 보여 실제보다 더 full/empty로 판단 → 절대 overflow/underflow 안 나는 안전한 pessimism. - 🧠 메모리 브릿지: 코어 clock ↔ 메모리 clock 사이 데이터 버퍼링(rate matching)의 표준 구조.

Q7. req/ack 핸드셰이크 동기화를 설명하세요. 언제 쓰나요?

A. 송신측이 데이터를 안정시킨 채 req=1 → 수신측이 req를 2-FF 동기화해 보고 데이터를 캡처한 뒤 ack=1 → 송신측이 ack를 동기화해 보고 req=0 → 수신측 ack=0 (4-phase). 데이터 버스는 req가 안정적으로 인식될 때까지 변하지 않으므로 동기화 불필요(false path). FIFO처럼 throughput이 필요 없고 가끔 다비트 control을 확실히 넘길 때 적합합니다. 단점은 왕복 동기화 지연으로 느립니다. - L, 꼬리질문: 2-phase와 4-phase 차이? → 2-phase는 레벨 toggle(엣지)로 한 번 왕복에 한 전송, 빠르지만 상태 추적 필요. 4-phase는 매번 0으로 복귀, 단순·robust. - 🧠 메모리 브릿지: 저빈도 mode change-trigger를 코어 → PHY로 확실히 넘길 때.

Q8. MUX-recirculation(data-hold)과 shadow register는 무엇인가요?

A. 다비트 데이터는 동기화하지 않고 도메인 경계 MUX의 한 입력에 그대로 두고, **enable 1-bit만 동기화**합니다. enable이 떨어진 동안은 캡처 FF가 자기 값을 되먹임(recirculation)해 유지하고, 동기화된 enable이 떴을 때만 새 데이터를 load. 데이터는 enable이 인식되는 순간 이미 안정돼 있으므로 안전하고 false-path 처리합니다. shadow register는 본 레지스터 옆에 그림자 레지스터를 두고 "valid 펄스 동기화 시점"에 한꺼번에 복사해 일관성을 보장하는 같은 계열 기법입니다. - L, 꼬리질문: enable 동기화만으로 데이터 안정이 어떻게 보장되나? → enable이 송신측에서 데이터보다 충분히 늦게 뜨도록 설계(데이터 먼저 안정 → 그 다음 enable). 정적 검증에서 이 가정을 assertion으로 확인. - 🧠 메모리 브릿지: PHY training 결과값(다비트)을 코어가 한꺼번에 읽을 때 valid 펄스+MUX load.

Q9. 빠른 → 느린 도메인으로 1-cycle 펄스를 보내면 왜 유실되나요? 해결책은?

A. 펄스 폭이 수신 clock 주기보다 좁으면 수신측이 한 번도 sampling 못 해 통째로 놓칠 수 있습니다. 해결은 **toggle 동기화기**: 송신측에서 펄스마다 레벨을 toggle → 그 레벨을 2-FF 동기화 → 수신측에서 edge detect(XOR)로 다시 펄스 복원. 레벨은 유실되지 않으므로 안전합니다. 단, 다음 toggle까지 최소 간격(수신 2~3 cycle) 보장 필요. - L, 꼬리질문: 펄스가 연속으로 들어오면? → toggle 간격이 수신 주기보다 짧으면 또 유실 → 사이에 FIFO나 handshake로 backpressure. - 🧠 메모리 브릿지: 저속 관리 clock에서 발생한 event를 고속 PHY로(혹은 반대로) 전달할 때.

Q10. quasi-static / false path / reconvergence를 CDC 관점에서 설명하세요.

A. **quasi-static**: 거의 변하지 않는 신호(설정값 등). 동작 중 안정적이라 static처럼 다루지만, 바뀌는 순간엔 여전히 안정-when-sampled를 보장해야 합니다. **false path**: STA에서 타이밍을 잡지 않는 경로 — 비동기 crossing의 데이터 버스, recirculation 데이터 등을 set_false_path로 제외. **reconvergence**: 한 소스에서 갈라져 따로 동기화된 신호들이 다시 합쳐질 때, 도착 cycle 차이로 glitch/불일치 발생 → 반드시 "한 번만 동기화 후 fan-out" 하거나 gray/handshake로 묶어 회피. - L, 꼬리질문: false path를 잘못 걸면? → 실제 타이밍 위반을 놓침. 그래서 CDC 도구로 "그 경로가 진짜 동기화기 뒤에 있는지" 구조 검증 후 waiver. - 🧠 메모리 브릿지: 다비트 config를 비트별 동기화 후 한 로직에서 합치면 reconvergence glitch → 한 번에 load해야.

Q11. CDC 정적 검증 방법론(SpyGlass/Questa)과 waiver를 설명하세요. (★지원서연계)

A. CDC sign-off는 (1)구조 검증: 모든 clock crossing을 자동 탐지하고 각 경로에 적절한 동기화기(2-FF/FIFO/handshake)가 있는지, 다비트는 gray/MUX 구조인지 확인, (2)기능 검증: 동기화 프로토콜·data-stability를 SVA assertion으로 생성해 시뮬/formal로 검증, (3)reset domain까지 본 RDC, (4)waiver: 안전하다고 입증된 crossing(false path, quasi-static)에 근거를 달아 violation을 정식 제외하고 재발 방지로 관리. 저는 실제로 Lint/CDC/RDC sign-off 방법론과 EDA 자동화를 만들며 도구 셋업·waiver 정책·noise 감축을 담당했습니다. - L, 꼬리질문: waiver 남발의 위험은? → 진짜 버그를 숨김. 그래서 waiver는 reason-coded·리뷰·버전관리하고, 자동화로 "신규 vs 기존 waiver"를 구분해 회귀 차단. - 🧠 메모리 브릿지: 메모리 IP는 도메인 수가 많아(코어/PHY/관리/test) crossing이 폭증 → 자동화된 CDC sign-off가 곧 품질.

Q12. SFR/CSR 설정값이 HW로 전달되는 path를 어떻게 동기화하나요? (★지원서연계·실제 업무)

A. SW가 버스 clock(예: APB)에서 CSR을 write하면, 그 다비트 설정이 다른 clock 도메인의 datapath/PHY로 넘어갑니다. 두 갈래로 나눕니다. (1)quasi-static config: 동작 중 거의 안 바뀌고 1비트씩 독립적으로 쓰이면 각 비트 2-FF + false-path. (2)다비트가 한꺼번에 의미를 갖는 config(예: mode, divider 값): bit skew가 치명적이라 "config update 펄스"를 동기화해 그 시점에 전체 비트를 MUX-load(recirculation)하거나 shadow register로 한꺼번에 복사. 추가로 write 중 잘못 읽히지 않게 SW가 enable을 마지막에 set하는 순서를 강제하고, CDC 도구로 이 구조를 assertion 검증·waiver합니다. - L, 꼬리질문: 읽기(status) 경로는? → HW status를 SW가 읽을 땐 반대 방향 동기화. 다비트 status는 capture-and-hold 후 valid 펄스 동기화로 한 스냅샷을 읽게 함. - 🧠 메모리 브릿지: 메모리 PHY의 training/calibration 파라미터를 register로 설정하는 길이 정확히 이 SFR→PHY-clock CDC. 신입이라도 "설정 path도 CDC"라고 말하면 강한 인상.

30초 암기 요약

- 메타스터빌리티는 못 없애고 MTBF로 관리. $MTBF = e^{(t_r/\tau)/(T_0 \cdot f_c \cdot f_d)}$, t_r 에 지수 의존.
- 2-FF 동기화기는 **1-bit-level 전용**. 두 FF 사이 로직 금지.
- 다비트는 bit skew 때문에 비트별 2-FF 금지 → Gray / async FIFO / handshake / MUX-recirculation.
- Gray=인접 1비트만 변함 → FIFO 포인터. async FIFO=Gray 포인터 2-FF 동기화로 full/empty.
- handshake=req/ack 4-phase, 느리지만 robust. MUX-recirculation=enable만 동기화, 데이터는 hold+false-path.
- 빠른→느린 펄스는 유실 → toggle 동기화기(level+edge detect).
- reconvergence/quasi-static/false-path 주의. 따로 동기화 후 재합류 금지.
- 정적 검증=SpyGlass/Questa CDC(구조+기능 SVA)+RDC+근거 있는 waiver.
- **SFR→HW config도 CDC**: 다비트 config는 update 펄스 동기화 후 한꺼번에 load (★내 실제 업무).